

项目二：计算机面试题 RAG 智能辅导助手 — 技术文档

一、项目概述

GitHub 地址：<https://github.com/officewh1/interview-rag-agent>

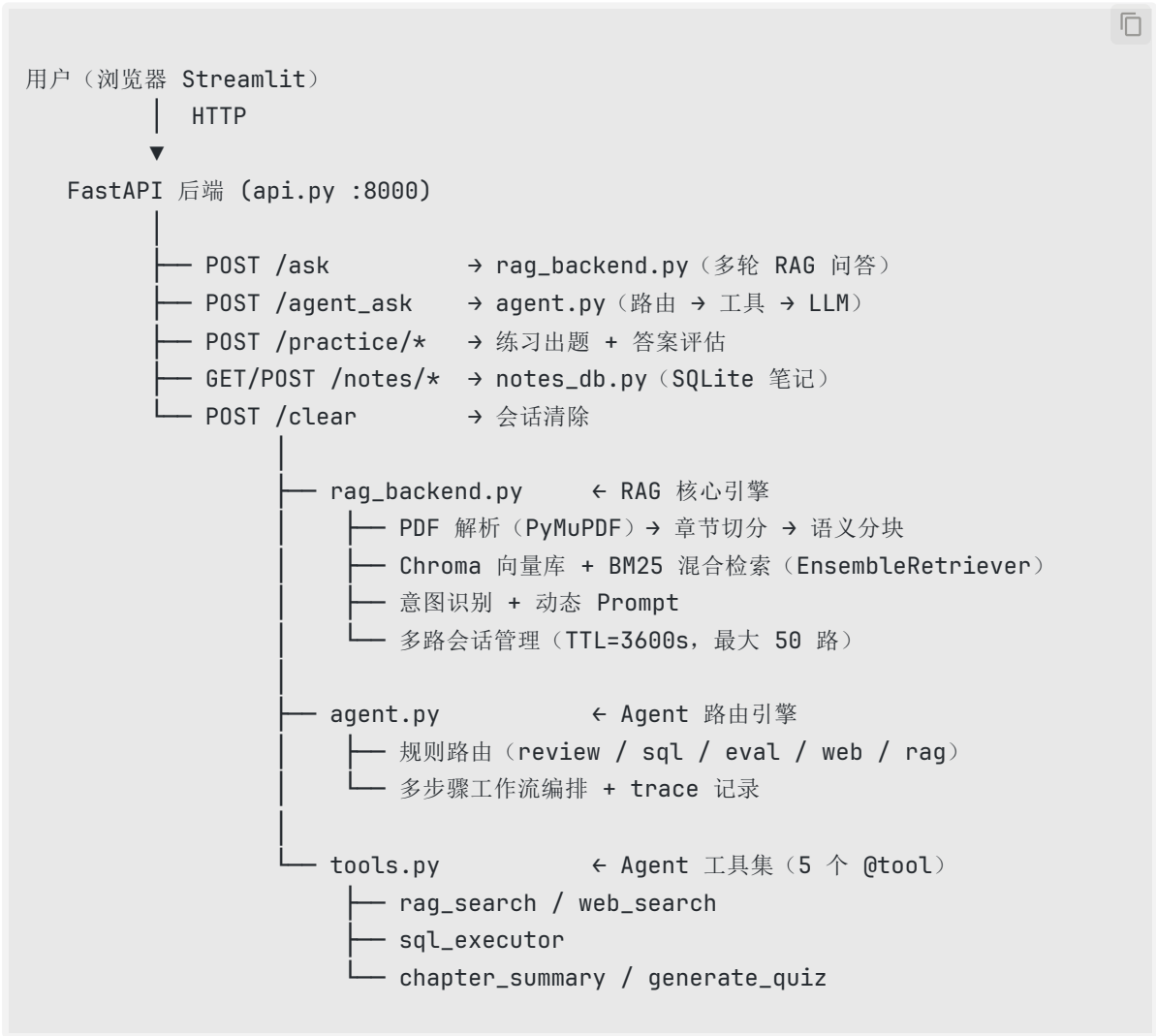
本项目是一个基于 LangChain + Ollama 本地部署的中文 RAG + Agent 系统，面向计算机专业面试备考，覆盖软件工程、数据库、机器学习、大数据四个方向。系统完全本地运行，无需外部 API。



核心功能：

- **混合检索**：BM25 (jieba 中文分词) + 向量检索 (bge-small-zh-v1.5) 融合，召回率更高
- **意图识别**：自动识别 definition / method / comparison / general 四种问题类型，动态调整 Prompt
- **多轮对话**：ConversationBufferMemory 实现上下文记忆，支持代词追问 ("它是什么?")
- **Agent 路由**：5 条规则路由自动分发，支持章节复习 workflow、SQL 沙箱、答案评估、Web 搜索
- **练习模式**：逐题作答 + LLM 出具详细评分报告 (评分依据 / 优点 / 不足 / 改进建议)
- **笔记系统**：一键保存任意 AI 回答，SQLite 持久化，按 session 隔离

二、系统架构



关键技术选型:

模块	技术选型	理由
大语言模型	Ollama + Qwen3-4B	本地推理，无需外部 API，temperature=0 保证稳定输出
嵌入模型	BAAI/bge-small-zh-v1.5	中文语义向量，约 184MB，完全离线
向量数据库	Chroma (持久化)	MD5 哈希版本管理，参数变更自动重建
关键词检索	BM25 + jieba	中文无空格，jieba 切词后 BM25 才能正常召回

模块	技术选型	理由
Agent 路由	规则路由（非 Function Calling）	Qwen3-4B 函数调用稳定性不足，规则路由路径固定、可追踪
Web 框架	FastAPI + Streamlit	前后端分离，前端支持展开 Agent trace

三、核心功能演示

3.1 章节复习 workflow

点击章节快捷按钮（如"软件工程"）后，Agent 三步串联执行：知识点总结 → 高频考点检索 → 生成模拟题，输出结构化复习报告。



- 核心知识点列表（5 条，含非功能性需求、开闭原则、单一职责、内聚与耦合）
- 模拟面试题（Q/A 格式，类型覆盖定义类、方法类、对比类）
- 侧边栏可切换 RAG / Agent 模式，章节快捷按钮一键触发

3.2 答案评估

用户提交针对面试题的答案后，系统通过 eval 路由检索参考答案，LLM 输出结构化评估报告。



- **优点：**识别回答中正确、完整的部分
- **不足 / 遗漏：**指出缺少的关键知识点（如未强调"软件怎么做"视角）
- **改进建议：**结合知识库参考答案给出完整补充说明

评估报告底部提供「查看 Agent 执行步骤」和「查看检索证据」两个可折叠面板，完整还原路由决策与检索过程；右侧"已保存为笔记"说明一键保存已生效。



3.3 练习模式

选择章节与题目数量后点击「开始练习」，系统逐题呈现面试题并接收用户作答，提交后即时生成详细评估报告。

数据库

机器学习

大数据

练习模式

软件工程

第 1 / 3 题

结束练习

我的笔记

★ 优点

> 面试官准确地指出了非功能性需求是软件产

会话控制

会话 ID: b2b8b15d...

清除对话

练习模式 · 软件工程

第 1 题 / 共 3 题

你的答案

是指软件产品为满足用户需求而必须具有且除功能需求以外的特性，包括安全性、可靠性、互操作性、健壮性等

评估报告

评估报告：软件非功能性需求的回答

作为一位严格但友善的软件工程评估者，我将基于您的回答（“软件的非功能性是指软件产品为满足用户需求而必须具有且除功能需求以外的特性，例如安全性、可靠性、互操作性、健壮性等”）进行客观分析。评估依据是软件工程标准（如IEEE 830标准）和知识库中的权威定义（来自您提供的上下文：非功能性 = 软件“怎么做”（如响应时间、安全性、可扩展性））。题目要求“什么是软件的非功能性？请举例说明”，因此我会重点关注：定义准确性、举例的完整性、表述的清晰度。

1. 综合评分：6/10

评分依据（满分10分）：

定义准确性（3分）：您正确区分了非功能性与功能需求（“除功能需求以外的特性”），并列举了常见非功能性特性（安全性、可靠性、互操作性、健壮性），这部分得2分（满分3分）。

举例完整性（3分）：题目要求“举例说明”，但您仅列出特性名称（如“安全性”），未提供具体、量化的例子（如响应时间、可扩展性），elicit 1分（满分3分）。：表述中“为满足用户需求”稍显模糊（非功能性更侧重软件的内在质量，而非直接业务需求），扣1分（满分2分）。

为什么是6分？

您的回答在核心概念上基本正确，但缺少具体例子是主要扣分点（题目明确要求“举例”）。在软件工程中，非功能性需求必须通过可测量的实例来体现（例如响应时间、安全性指标），而非仅抽象描

- 综合评分（满分 10 分）附评分依据分项说明
- 扣分点逐条说明（如举例完整性不足、表述偏差）
- 改进方向给出参考标准答案

四、模块详解

4.1 RAG 核心引擎 (rag_backend.py)

数据处理流水线：

PDF 文件

↓ PyMuPDF 逐页提取 + 正则清洗

原始文本

↓ 按章节标题切分（软件工程 / 数据库 / 机器学习 / 大数据）

4 个章节文本块

↓ RecursiveCharacterTextSplitter

chunk_size=500, overlap=80, min_len=50

分隔符：句号 / 分号 / 换行

文本 chunks（附 chapter + chunk_type 元数据）

└─ bge-small-zh-v1.5 → Chroma 向量库（持久化 + MD5 版本管理）

└─ jieba 分词 → BM25 索引（pickle 缓存，启动加速）

混合检索：

检索器	权重	算法
BM25Retriever	0.4	TF-IDF 词频统计，jieba 分词 + 停用词过滤
Chroma 向量检索	0.6	余弦相似度，bge-small-zh-v1.5 编码

两路各取 top-3，EnsembleRetriever 按权重融合后去重排序。

意图识别与动态 Prompt：

类型	触发关键词	Prompt 指令
definition	什么是、定义、含义、概念	先给出清晰定义，再说明关键特征
method	如何、怎么、步骤、流程	按步骤清晰说明方法或流程
comparison	区别、对比、优缺点、比较	分点对比异同与优缺点
general	其他	基于参考内容准确简洁地回答

多轮会话管理：ConversationBufferMemory，session_id 隔离，最多 50 路并发，TTL 3600 秒自动清除，Chain 按 (session_id, q_type) 缓存避免重复构建。

4.2 Agent 路由引擎 (agent.py) ..

使用规则路由替代 LLM Function Calling (Qwen3-4B 小参数模型函数调用稳定性不足)，每条路径固定，所有步骤均有 trace 记录。

路由优先级：

优先级	路由	触发条件
1	review	含"复习 / 总结 / 梳理 / 考点 / 模拟题"等
2	sql	含 SELECT / INSERT / CREATE 等 SQL 语句
3		含"我的答案 / 我认为 / 评分 / 打分"等

优先级	路由	触发条件
	<code>![]</code> <code>(http://localhost:63342/markdownPreview/688148641/commandRunner/run.png)eval</code>	
4	<code>web</code>	含"最新 / 趋势 / 2025 / 现状"等时效性词
5	<code>rag</code>	其他（默认）

review 路由三步 workflow:

```

① chapter_summary → 检索 5 条文档，生成 5 个核心知识点
    ↓
② rag_search → 查询该章节高频考点
    ↓
③ generate_quiz → 生成 3 道模拟题（Q/A 格式，含参考答案）
    ↓
组合输出：知识总结 + 模拟题 Markdown 报告

```

4.3 Agent 工具集 (tools.py) ..

工具	功能
<code>rag_search</code>	混合检索本地知识库，支持按章节过滤，返回 top-3 文档片段
<code>web_search</code>	DuckDuckGo 搜索（中文区），返回最多 3 条互联网结果
<code>sql_executor</code>	SQLite 内存沙箱执行 SQL，含安全拦截黑名单
<code>chapter_summary</code>	三路检索 5 条文档，LLM 生成结构化知识总览
<code>generate_quiz</code>	检索 3 条文档，LLM 生成 n 道 Q/A 模拟题

4.4 SQL 沙箱 (sql_executor.py) ..

每次调用 `sqlite3.connect(":memory:")` 创建全新内存库，调用结束自动销毁，不写入任何本地文件。预置 6 张测试表：

表名	适用考题场景
employees / departments	薪资查询、自连接、多表 JOIN
students / courses / enrollments	GROUP BY 聚合、子查询
orders	日期过滤、统计分析

五、API 接口

方法	路径	功能
POST	/ask	多轮 RAG 问答 (含会话记忆)
POST	/agent_ask	Agent 模式 (路由 + trace + sources)
POST	/clear	清除指定会话
POST	/practice/questions	生成练习题目列表 (优先走缓存)
POST	/practice/evaluate	评估练习答案
POST	/notes/save	保存笔记
GET	/notes	查询笔记列表
DELETE	/notes/{note_id}	删除笔记

/agent_ask 返回字段: answer / question_type / route / used_web / sql_result / sources / trace

六、部署指南

环境要求: Python 3.9+, Ollama, 内存 ≥ 8GB, 磁盘 ≥ 4GB

```
# 1. 安装依赖
pip install -r requirements.txt

# 2. 下载嵌入模型
huggingface-cli download BAAI/bge-small-zh-v1.5 --local-dir ./bge-small-zh-v1.5

# 3. 启动 Ollama
ollama serve
```



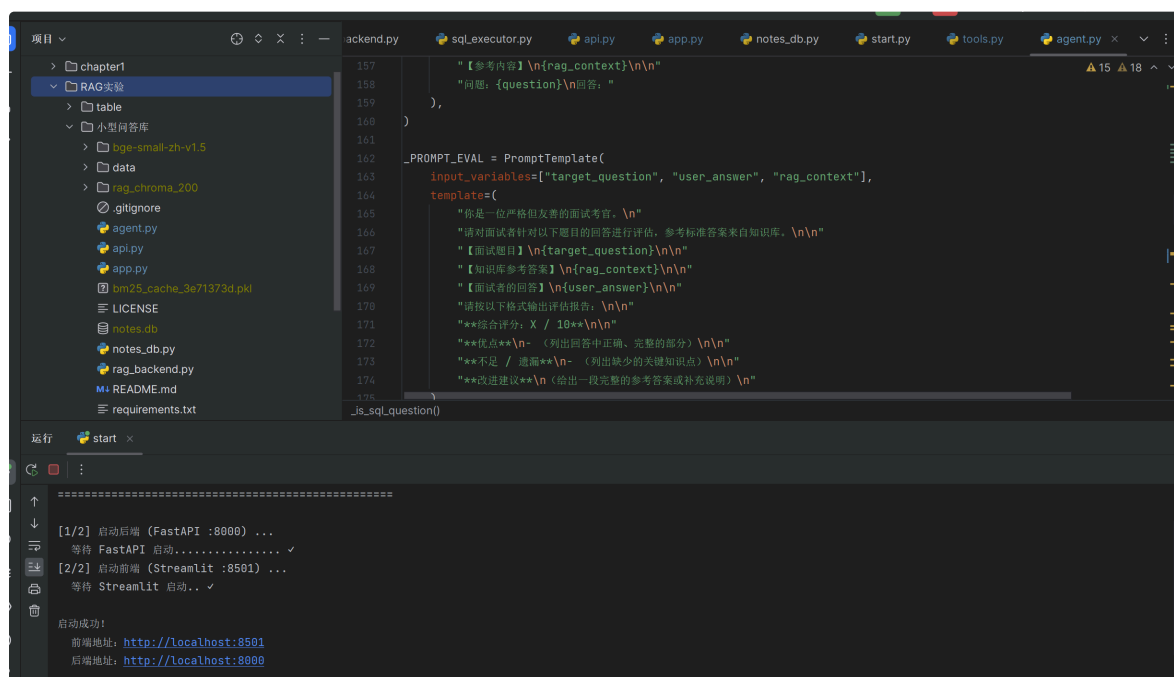
```
ollama pull qwen3:4b

# 4. 放入 PDF (data/章节面试题.pdf, 需含四个章节标题)

# 5. 一键启动
python start.py
```

浏览器自动打开 <http://localhost:8501> , Ctrl+C 停止所有服务。

下图为 IDE 中的项目文件结构与终端启动日志, FastAPI (:8000) 与 Streamlit (:8501) 均已成功拉起:



首次启动耗时约 **1~3 分钟** (PDF 解析 → 向量库构建 → BM25 索引) , 后续启动参数未变更时复用缓存, 耗时约 **10~20 秒**。

七、已知限制

- Qwen3-4B 本地推理速度受 CPU/GPU 配置影响, 复习工作流 (三步串联) 响应时间约 60~120 秒
- 题目预缓存在后台线程生成, 服务重启后首次练习仍需等待约 30~60 秒
- Web 搜索依赖 DuckDuckGo 连通性, 网络不稳定时降级为纯 RAG 回答
- 向量库与 BM25 索引基于 PDF 内容构建, 知识范围受 PDF 质量限制

- ConversationBufferMemory 保留全量历史，长对话可能超出 num_ctx=2048 上限
-

文档结束